

🎵 AI Music Generation with LSTM

MIT 6.S191 Deep Learning Lab Project — Generating original music using Recurrent Neural Networks (LSTM) trained on Irish folk songs in ABC notation.

🎬 Demo

Click the badge above to open the notebook directly in Google Colab — no setup required!

What You'll Get

- 🎵 **Original music** generated by AI
- 🔊 **Audio playback** inside the notebook
- 💾 **WAV file export** for download
- ⚡ **GPU-accelerated** training

📄 Table of Contents

- Overview
- Architecture
- Quick Start
- Project Structure
- How It Works
- Results
- Technologies
- License

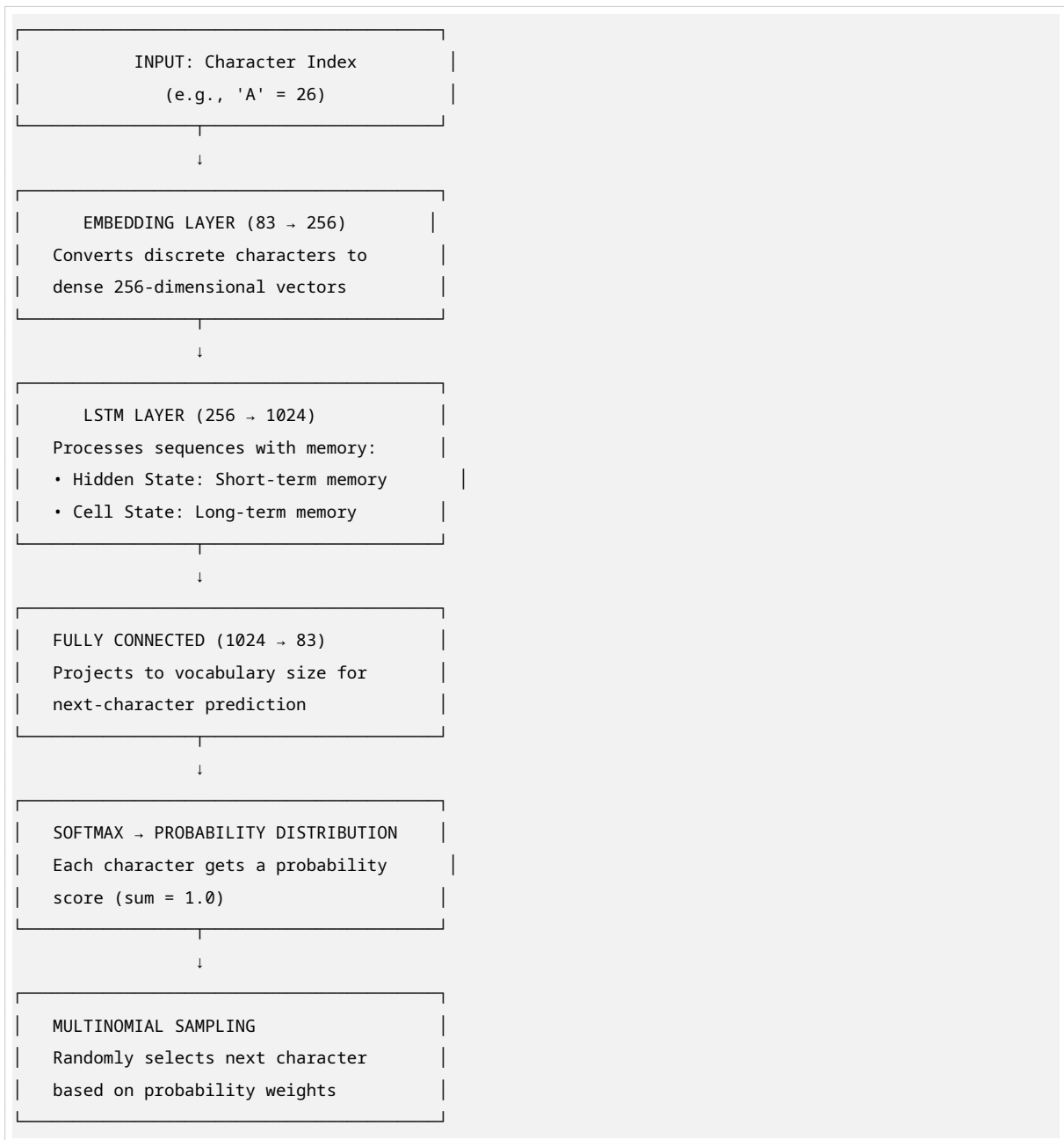
🎯 Overview

This project implements a **character-level LSTM (Long Short-Term Memory)** network that learns patterns from Irish folk music encoded in [ABC notation](#). The trained model can generate new, original musical compositions character by character.

Key Features

Feature	Description
🧠 LSTM Architecture	1024 hidden units with memory for long musical sequences
📝 ABC Notation	Text-based music representation (A-G notes, rhythm symbols)
🎛️ Temperature Sampling	Adjustable creativity: conservative (0.5) to experimental (1.5)
🔊 Audio Synthesis	Automatic conversion to playable WAV files
📊 Loss Tracking	Real-time training visualization

🏗️ Architecture



Model Specifications

Parameter	Value	Description
vocab_size	83	Unique characters in ABC notation
embedding_dim	256	Dimension of character embeddings
hidden_size	1024	LSTM hidden units
batch_size	32	Sequences per training batch

Table 2 – continued

Parameter	Value	Description
seq_length	100	Characters per sequence
learning_rate	5e-3	Adam optimizer learning rate
iterations	3000	Training iterations
Total Parameters	~2.4M	Trainable weights

Quick Start

Option 1: Google Colab (Recommended — Zero Setup)

1. Click the “**Open in Colab**” badge at the top
2. Go to **Runtime** → **Run all** (Ctrl+F9)
3. Scroll to the “**Music Generation**” section
4. Customize parameters and generate music!

Option 2: Local Setup

```
# Clone the repository
git clone https://github.com/YOUR_USERNAME/music-generation-mit.git
cd music-generation-mit

# Install dependencies
pip install torch numpy scipy tqdm mitdeeplearning

# Install audio tools (Linux/Mac)
apt-get install abcmidi timidity ffmpeg

# Run the notebook
jupyter notebook music_generation_rnn.ipynb
```

Project Structure

```
music-generation-mit/
|
├── music_generation_rnn.ipynb    # Main notebook (Colab-ready)
├── README.md                    # This file
├── requirements.txt              # Python dependencies
├── generated_songs/             # Output WAV files (generated)
│   ├── generated_song_1.wav
│   └── generated_song_2.wav
└── training_checkpoints/        # Model weights (saved during training)
    └── model.pt
```

How It Works

1. Data Preprocessing

- Loads Irish folk songs in ABC notation
- Builds vocabulary of 83 unique characters
- Vectorizes text into numerical indices
- Creates input-output pairs for next-character prediction

2. Training Process

```
For each iteration:
1. Sample random sequences from dataset
2. Forward pass: Model predicts next character
3. Calculate Cross-Entropy Loss
4. Backpropagation: Compute gradients
5. Adam Optimizer: Update weights
6. Log metrics and save checkpoints
```

3. Music Generation

```
For each character to generate:
1. Feed current sequence to LSTM
2. Get probability distribution over vocabulary
3. Apply temperature scaling
4. Sample next character using multinomial distribution
5. Append to output, feed back as new input
6. Repeat until desired length reached
```

Temperature Explained

Temperature	Behavior	Use Case
0.5	Conservative, repetitive	Structured melodies
1.0	Balanced	Natural-sounding music
1.5	Creative, experimental	Unique compositions

Formula: $\text{logits} = \text{logits} / \text{temperature}$

- Lower temp → sharper distribution → predictable
- Higher temp → flatter distribution → random

Results

Training Loss

The model converges over 3000 iterations with decreasing cross-entropy loss:

```
Iteration 0:    Loss ≈ 4.4 (random guessing)
Iteration 500:  Loss ≈ 2.1 (learning patterns)
Iteration 1500: Loss ≈ 1.5 (recognizing structure)
Iteration 3000: Loss ≈ 0.8 (generating coherent music)
```

Generated Music Examples

The model produces valid ABC notation that can be:

-  Parsed into structured songs
-  Synthesized into audio waveforms
-  Exported as WAV files
-  Played directly in the notebook

Technologies

Technology	Purpose
PyTorch	Deep learning framework
LSTM	Sequence modeling with memory
CrossEntropyLoss	Multi-class classification
Adam Optimizer	Adaptive gradient descent
ABC Notation	Text-based music representation
MIDI/Timidity	Audio synthesis
Google Colab	Cloud GPU training

Learning Outcomes

Through this project, I gained hands-on experience with:

-  **RNNs & LSTMs** — Understanding memory in neural networks
-  **Text Generation** — Character-level sequence modeling
-  **Embedding Layers** — Converting discrete data to dense vectors
-  **Temperature Sampling** — Controlling randomness in generation
-  **Audio Processing** — Converting text notation to sound
-  **Experiment Tracking** — Logging metrics with Comet ML

Links

-  **Course:** [MIT 6.S191 Introduction to Deep Learning](#)
-  **ABC Notation:** abcnotation.com
-  **Dataset:** Irish Traditional Music Archive
-  **PyTorch Docs:** pytorch.org/docs

License

This project is licensed under the MIT License — see the [LICENSE](#) file for details.

Author: [Your Name]

Course: MIT 6.S191 Deep Learning

Year: 2024

★ Star this repo if you found it helpful!